

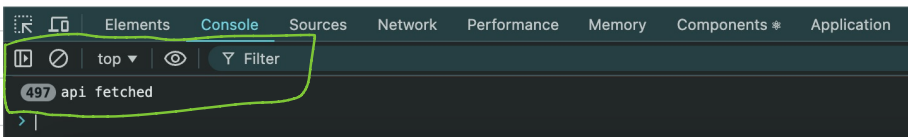

Lec 7 2

2 Jan 2025

Samarth Sir

localhost:5173

api call kr rha hu



```
import React, { useState } from "react";
```

```
function Effects() {  
  let [d, setD] = useState("");
```

```
    fetch("https://api.tvmaze.com/search/shows?q=girls").then(async  
function (   
  res  
  ) {  
    let data = await res.json();  
    console.log("api fetched");  
    setD(data);  
  });
```

```
  return <div>api call kr rha hu </div>;  
}
```

```
export default Effects;
```

upar wala code loop mei fass gaya kyu?
to baar baar chalta rahega kyu?

The issue with your code is that the fetch call is being executed every time the component renders. React components re-render when the state or props change, and since the `setD(data)` updates the state, it triggers a re-render, which in turn re-executes the fetch call. This creates an infinite loop of renders and fetches.

How to Fix the Issue

To prevent this, you should use the `useEffect` hook. The `useEffect` hook allows you to perform side effects like fetching data and can control when the side effect runs by specifying dependencies.

Fixed Code

jsx

Copy code

```
import React, { useState, useEffect } from "react";

function Effects() {
  const [d, setD] = useState("");

  useEffect(() => {
    // Fetch data only once when the component mounts
    fetch("https://api.tvmaze.com/search/shows?q=girls")
      .then((res) => res.json())
      .then((data) => {
        console.log("api fetched");
        setD(data);
      })
      .catch((err) => console.error("Error fetching data:", err));
  }, []); // Empty dependency array ensures this runs only once

  return <div>api call kr rha hu</div>;
}

export default Effects;
```

Explanation

1. `useEffect` Hook:

- By wrapping the `fetch` call inside `useEffect`, you ensure it runs only when the component mounts.

2. `Dependency Array []`:

- The empty array as the second argument to `useEffect` means the effect runs only once, similar to `componentDidMount` in class components.

3. `State Update`:

- `setD(data)` updates the state without causing infinite re-renders because `useEffect` is not re-triggered unless its dependencies change.

This ensures that the `fetch` call doesn't keep running unnecessarily.

Now we will use `axios` instead of `fetch`. Install it "npm i axios" in "hooks - more" folder.

```
import React, { useEffect, useState } from "react";
import axios from "axios";

function Right() {
  let [d, setD] = useState('');

  ➔useEffect(function(){
    axios.get("https://api.tvmaze.com/search/shows?q=girls")
      .then(function(res){
        console.log(res);
        setD(res); ➔
      })
    console.log("render"); ➔
  }, []) ➔ 22

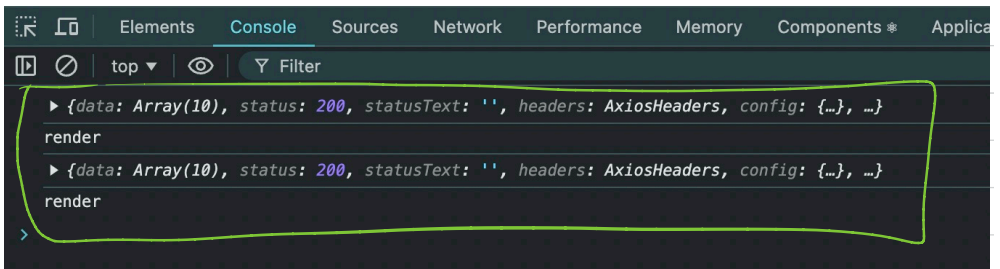
  return (
    <div>
      Right
    </div>
  )
}

export default Right
```

➔ Right.jsx

localhost:5173

Right



```
▶ {data: Array(10), status: 200, statusText: '', headers: AxiosHeaders, config: {...}, ...}  
render  
▶ {data: Array(10), status: 200, statusText: '', headers: AxiosHeaders, config: {...}, ...}  
render  
>
```

↳ yaha o/p mei loop nahi aayi kyunki hamne use Effect use kiya. (Infinite sender nahi huye)

Infinite sending isliye bhi nahi huyi kyunki array khaali hai 2 par.

```

import React, { useEffect, useState } from "react";
import axios from "axios";

function Right() {
  let [d, setD] = useState('');

  useEffect(function(){
    axios.get("https://api.tvmaze.com/search/shows?q=girls")
      .then(function(res){
        console.log(res);
        setD(res);
      })
      console.log("render");
  }, [d]) → d

  return (
    <div>
      Right
    </div>
  )
}

```

→ Right.jsx

→ Humne d par d write kar diya; ab infinite rendering hogi.

d is a dependency array. When `setD(res)` updates the state d, it triggers a re-render.

The `useEffect` runs whenever a dependency in its dependency array changes.

In your case, `d` is in the dependency array. When `setD(res)` updates the state `d`, it triggers a re-render.

During the re-render, the `useEffect` runs again because `d` has changed.

This creates an infinite loop because every time `d` is updated, the `useEffect` runs and updates `d` again.

Note:- leave the dependency array empty if you have not used the state variable inside `useEffect()`.

use Memo :- (It is a hook)

The useMemo hook in React is used to optimize performance by memoizing the result of a computation. It prevents expensive calculations from being re-executed unnecessarily during every render of a component. Instead, it recomputes the result only when the dependencies change.

using loop

Mean do mujhe 1 to n tak ka sum nikalna hai -
that is 5050.

Somehow state change hogi, then component re render hoga and loop firse evaluate hogi. Then state firse change hogi; ~~re~~ loop firse chalegi. We noticed ki loop baar baar chal rahi hai. Efficiency kam hogi, ise avoid karna padega. Baar baar evaluate mat kro, yaad rakh lo value. This is where useMemo() comes to solve.

Agar $n=104$ ho jaaye from $n=100$; tab to evaluate karna hi padega; loop chalegi fir.

import React, { useState } from 'react'

function RightMemo() {

let [inp, setInp] = useState(0);

let [count, setCount] = useState(0);

function handleChange(e){
setInp(e.target.value);
}

function handleClick(){
setCount(count+1)
}

let ans=0;

for(let i =1; i<= inp; i++){
ans= ans+i;
console.log('loop');
}

→ 11

return (

<div>

<input onChange = {handleChange} type="number" value = {inp}/>

<h1>Sum till that number: {ans}</h1>

<button onClick ={handleClick}>Counter - {count}</button>

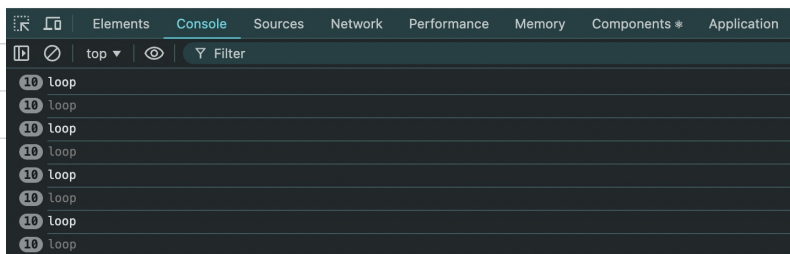
</div>

)

}

export default RightMemo

→ RightMemo.
jsx



For the code given above, sabse pehle maine 10 type kiya tab loop 10 baar chali.

Fir maine button par click kiya; then loop 10 baar chali. Jitni baar maine button par click kiya, utni baar loop chali jabki input mei maine value bhi change nahi kari. Fir bhi loop chale ja raha hai sum ka loop. we will need to fix it.

11 ko convert kar do useMemo mei.

Iss code mei "loop" ka kaam hai evaluate ki gyi value ko yaad rakhna/sambhal kar rakhna.

useMemo (cbfunc, [])
↳ dependency array

apna logic cb function mei paste kardo.

```
let loopp = useMemo(  
  function () {  
    let ans = 0;  
    for (let i = 1; i <= inp; i++) {  
      ans = ans + i;  
      console.log("loop");  
    }  
    return ans;  
  },  
  [inp] → (l2)  
);
```

(l1)

(l2)

Note - kyunki hamne "inp" state variable use kiya "useMemo" mei isliye use likho inside array (l2)

Also, inp ki value change hone par loop function fir chalega.

```
import React, { useMemo, useState } from "react";
```

```
function MyMemoHook() {  
  let [count, setCount] = useState(0);  
  let [inp, setInp] = useState(0);  
  function handleClick() {  
    setCount(count + 1);  
  }  
  function handleChange(e) {  
    setInp(e.target.value);  
  }  
  
  let loopp = useMemo(  
    function () {  
      let ans = 0;  
      for (let i = 1; i <= inp; i++) {  
        ans = ans + i;  
        console.log("loop");  
      }  
      return ans;  
    },  
    [inp]  
  );  
  return (  
    <div>  
      <input onChange={handleChange} type="number" value={inp} />  
      <h1>Sum till that number: {loopp}</h1>  
      <button onClick={handleClick}>Counter - {count}</button>  
    </div>  
  );  
}
```

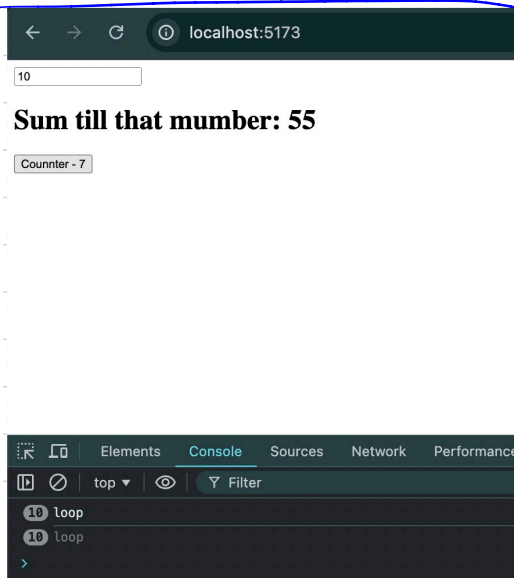
→ 11

l3 →

→ 12

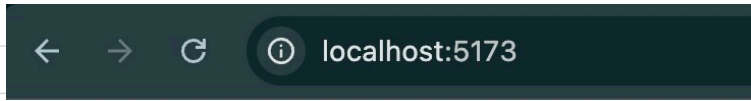
```
export default MyMemoHook;
```

→ Right Memo.jsn



→ maine jab 10 likha
jab 10 loops chali.
But jab counter par
kuch baar click kiya,
then loop nahi
chali kuki {inp} ki
value change nahi
huyi.

Q1 sabse pehle tab chalega jab { loop } evaluate hoga Q2 par. And initially { loop } tab hoga jab return ka poora code chalega (Q3) isliye initially ye zero hai.



0

Sum till that number: 0

Counter - 0

Q1 par initially $i = 1$ and $inp = 0$

$i \leq 0 \rightarrow$ false, isliye $ans = 0$
return ho jayega.

par jab inp ki value change hogi tab tab { loop } reevaluate hoga.

code explanation :-

1. State Variables

The component uses two `useState` hooks to manage state:

- `count` : Tracks how many times the "Counter" button is clicked.

```
js Copy code  
  
let [count, setCount] = useState(0);
```

- `inp` : Tracks the value entered in the input field.

```
js Copy code  
  
let [inp, setInp] = useState(0);
```

2. Event Handlers

- `handleClick` : Increments the `count` state by 1 when the button is clicked.

```
js Copy code  
  
function handleClick() {  
  setCount(count + 1);  
}
```

- `handleChange` : Updates the `inp` state with the value entered in the input field.

```
js Copy code  
  
function handleChange(e) {  
  setInp(e.target.value);  
}
```

3. Computation with `useMemo`

The `useMemo` hook calculates the sum of numbers from 1 to `inp` and memoizes the result to avoid recalculating it unnecessarily.

- **Purpose:** The computation is expensive (it involves a loop), so `useMemo` ensures it only runs when the `inp` value changes.

js

 Copy code

```
let looppp = useMemo(  
  function () {  
    let ans = 0;  
    for (let i = 1; i <= inp; i++) {  
      ans = ans + i; // Calculates the sum  
      console.log("loop"); // Logs whenever the loop runs  
    }  
    return ans; // Returns the sum  
  },  
  [inp] // Dependency array: Only re-run this computation when `inp` changes.  
);
```

4. JSX Rendering

The component renders:

- An input field where the user can type a number. The `onChange` handler updates `inp`.
- A heading displaying the sum of numbers up to the entered value (`looppp`).
- A button that increments the `count` state and displays the current value.

jsx

 Copy code

```
return (  
  <div>  
    <input onChange={handleChange} type="number" value={inp} />  
    <h1>Sum till that number: {looppp}</h1>  
    <button onClick={handleClick}>Counter - {count}</button>  
  </div>  
);
```

Key Observations

1. Memoization with `useMemo`:

- The loop will only execute when `inp` changes.
- If you repeatedly click the button (changing `count`), the `loopp` value doesn't recalculate because `inp` hasn't changed.

2. Efficient Rendering:

- The `useMemo` optimization ensures the component doesn't recalculate the sum unnecessarily, saving processing time when the computation is expensive.

3. Logged Output:

- You'll see "loop" in the console only when the `inp` value changes, proving that the loop isn't executed unnecessarily.

Usage Example

1. Type `5` in the input field:

- `loopp` will calculate and display `15` ($1 + 2 + 3 + 4 + 5$).
- You'll see "loop" logged in the console 5 times.

2. Click the "Counter" button:

- `count` increases, but the sum doesn't recalculate (no additional "loop" logs appear).

↙ code execution during run time :-

Step 1: Component Initialization

1. The `MyMemoHook` component is mounted for the first time.

2. State Variables Initialized:

- `count = 0`
- `inp = 0`

3. Memoized Value (`loopp`):

- `useMemo` is invoked.
- The `inp` value is `0`, so the loop doesn't run, and `loopp` is set to `0`.

Step 2: Initial Rendering

1. The `return` statement is executed to render the UI:

- Input field with value `0`.
- Heading: `Sum till that number: 0`.
- Button: `Counter - 0`.

Step 3: User Interaction - Input Change

1. User types `5` in the input field:

- The `handleChange` function is triggered:

```
js
function handleChange(e) {
  setInput(e.target.value); // Updates `inp` to 5
}
```

- React re-renders the component because `inp` changed.

2. During re-render:

- `useMemo` Execution:
 - Since `inp` has changed (`[inp]` dependency), the memoized function runs.
 - Loop calculates the sum of numbers from `1` to `5`:

```
js
for (let i = 1; i <= inp; i++) {
  ans = ans + i; // 1 + 2 + 3 + 4 + 5 = 15
}
```

- Logs `"loop"` 5 times in the console.
- `loopp = 15` is returned.

3. Updated UI:

- Input field with value `5`.
- Heading: `Sum till that number: 15`.
- Button: `Counter - 0`.

Step 4: User Interaction - Button Click

1. User clicks the button:

- The `handleClick` function is triggered:

```
js Copy code  
  
function handleClick() {  
  setCount(count + 1); // Updates `count` to 1  
}
```

- React re-renders the component because `count` changed.

2. During re-render:

- `useMemo` Does Not Recalculate:
 - The `[inp]` dependency hasn't changed, so the memoized value (`loop = 15`) is reused.
 - The loop does not run, and no `"loop"` is logged in the console.

3. Updated UI:

- Input field with value `5`.
- Heading: `Sum till that number: 15`.
- Button: `Counter - 1`.

Step 5: User Changes Input Again

1. User changes the input value to 3 :

- The `handleChange` function updates `inp` to 3 .
- React re-renders the component.

2. During re-render:

- `useMemo` Execution:
 - Since `inp` has changed, the memoized function runs again.
 - Loop calculates the sum of numbers from 1 to 3 :

```
js
```

[Copy code](#)

```
ans = 1 + 2 + 3 = 6;
```

- Logs "loop" 3 times in the console.
- `loopp = 6` is returned.

3. Updated UI:

- Input field with value 3 .
- Heading: Sum till that number: 6 .
- Button: Counter - 1 .

let's go more deeper :-

Step-by-Step Explanation When Button is Clicked

1. Click Event:

- `handleClick` is triggered.
- `setCount(count + 1)` updates `count` .

2. Re-render:

- React re-executes the `MyMemoHook` component.
- `useMemo` :
 - Since `inp` didn't change, `loopp` retains its previous value.
- JSX is regenerated with the updated `count` .

3. DOM Update:

- React compares the new virtual DOM with the old one.
- It only updates the button text to display the new `count` .

Q what exactly happens when we put 5 in input field?

1. Input Change Event

- The `onChange` event of the `<input>` field triggers the `handleChange` function:

javascript

Copy code

```
function handleChange(e) {  
  setInput(e.target.value); // Updates `inp` to 5  
}
```

- The `setInp` function updates the `inp` state to 5.

2. React Schedules a Re-render

React detects the state change (`inp`) and re-executes the entire `MyMemoHook` function.

3. Execution During Re-render

Here's what happens step-by-step during the re-render:

a) State Re-initialization

- React does not reinitialize state variables. Instead, it uses the existing state values:
 - `count` remains the same (unchanged).
 - `inp` is now 5.

b) Recalculation with `useMemo`

- The `useMemo` hook is evaluated:

javascript

Copy code

```
let looppp = useMemo(  
  function () {  
    let ans = 0;  
    for (let i = 1; i <= inp; i++) {  
      ans = ans + i; // Adds numbers from 1 to 5  
      console.log("loop"); // Logs each loop iteration  
    }  
    return ans; // Returns 15  
  },  
  [inp] // Dependency array: Only re-runs when `inp` changes  
);
```

- Since `inp` has changed to `5`, the `useMemo` function re-executes:
 - The loop runs 5 times, logging `"loop"` in the console 5 times.
 - The result (`15`) is stored in `looppp`.

c) JSX Regeneration

- React regenerates the JSX based on the updated state and memoized value:

jsx

Copy code

```
return (  
  <div>  
    <input onChange={handleChange} type="number" value={inp} />  
    <h1>Sum till that number: {looppp}</h1>  
    <button onClick={handleClick}>Counter - {count}</button>  
  </div>  
);
```

- The following updates happen in the virtual DOM:
 - Input field: Reflects the new value (`5`).
 - Heading: Displays the updated sum (`15`).
 - Button: Remains unchanged (`Counter - 0`).

4. React DOM Update

- React compares the new virtual DOM with the old one.
- It identifies the changes:
 - Updates the value of the `<input>` to `5`.
 - Updates the `<h1>` to display `Sum till that number: 15`.
- The button (`Counter - 0`) remains the same, so it isn't updated.

What Re-renders?

1. Entire Component Function (`MyMemoHook`):
 - React re-executes the function to generate the new virtual DOM.
2. `useMemo` Hook:
 - Recalculated because the `inp` value changed.
3. Specific DOM Elements Updated:
 - `<input>` : Updates its value to `5`.
 - `<h1>` : Updates the displayed sum to `15`.
4. Unchanged Elements:
 - `<button>` : Since `count` didn't change, React skips updating this part of the DOM.

Summary

- When you enter `5`:
 1. The state variable `inp` is updated.
 2. `useMemo` recalculates the sum since `inp` changed.
 3. React regenerates the virtual DOM and compares it with the old one.
 4. Only the `<input>` and `<h1>` elements are updated in the actual DOM.
 5. The `<button>` remains untouched.

1:18:40

`useCallback()` → it is a hook.

→ ye bhi memoization karta hai but functions ko (naaki values ko).

→ it relies on the referential equality and avoids unnecessary renders.

`useCallback` is a React Hook that returns a memoized version of a function. It ensures that the function reference remains the same between re-renders unless its dependencies change.

```
> let a = 10
< undefined
> let b = 10
< undefined
> a === b → l1
< true
> let arr = [1, 2, 3]
< undefined
> let arr2 = [1, 2, 3]
< undefined
> arr === arr2 → l2
< false
> [1, 2, 3] === [1, 2, 3] → l3
< false
> |
```

arr and arr2 have same values but they are referentially different.

2 false hai kyunki memory ka address alag alag hai dono array ka.

2. Array Comparison

javascript

Copy code

```
let arr = [1, 2, 3];  
let arr2 = [1, 2, 3];  
arr === arr2; // false
```

• Why false?

- Arrays are **reference types** in JavaScript.
- `arr` and `arr2` are two separate array objects stored at different memory locations.
- When comparing objects (like arrays) with `===`, it checks if the references point to the same memory location, not their content.
- Since `arr` and `arr2` are two different objects, the result is `false`.

3. Direct Array Comparison

javascript

Copy code

```
[1, 2, 3] === [1, 2, 3]; // false
```

• Why false?

- Similar to the above example, these are two new array objects being compared.
- Even though they contain the same values, their memory references are different.
- As a result, `===` evaluates to `false`.

Conclusion

In summary, using `===` or `==` for object comparison in JavaScript checks if both variables point to the same memory address, not whether their contents are identical. This is crucial for understanding how JavaScript handles object comparisons and helps avoid common pitfalls when working with objects and arrays.

In the code snippet you provided:

javascript

```
let a = 10;  
let b = 10;  
console.log(a === b);
```



the `===` operator is used to compare two variables, `a` and `b`. Since both `a` and `b` are assigned the primitive value `10`, the comparison does not check memory references as it would for objects or arrays. Instead, it checks the actual values.

Key Points

1. **Primitive Values:** In JavaScript, `10` is a primitive value of type `Number`. Primitive values are immutable and do not have properties or methods associated with them. They are compared by their actual value rather than their reference in memory 2 5.

```
import React, { memo, useCallback, useState } from "react";
```

```
function Calll() {  
  let [count, setCount] = useState(0);  
  
  let a = 1;  
  
  return (  
    <div>  
      <Demo b={a} />  
      <button onClick={() => setCount(count + 1)}>Click {count} </button>  
    </div>  
  );  
}
```

```
function Demo({ b }) {  
  return (  
    <div>  
      <h1>hi = {b}</h1>  
    </div>  
  );  
}
```

```
export default Calll;
```

→ calll.jsx

→ is code mei figal ki re-rendering ho rahi hai kyunki button ke click hone par count change ho raha hai jiske karan Calle component re-render ho raha hai. How to prevent that?

```
import React, { memo, useCallback, useState } from "react";
```

```
function Calll() {  
  let [count, setCount] = useState(0);  
  
  let a = 1;  
  
  return (  
    <div>  
      <Demo b={a} />  
      <button onClick={() => setCount(count + 1)}>Click {count} </button>  
    </div>  
  );  
}
```

```
let Demo = memo( function({ b }) {  
  return (  
    <div>  
      <h1>hi = {b}</h1>  
    </div>  
  );  
})
```

```
export default Calll;
```

→ when count changes, only Calle re-renders, Demo does not bcoz prop passed to Demo is not getting changed.

```
let Demo = memo(function({ b }) {
  return (
    <div>
      <h1>hi = {b}</h1>
    </div>
  );
});
```

- `React.memo` is a higher-order component that wraps the `Demo` component.
- It memoizes the `Demo` component, meaning:
 - If the `props` passed to `Demo` do not change, React skips re-rendering `Demo`.
 - Here, the `b` prop is always `1`, so `React.memo` prevents unnecessary re-renders of `Demo` when the `count` state in the `Calll` component changes.

```
import React, { memo, useCallack, useState } from "react";

function Calll() {
  let [count, setCount] = useState(0);

  function a() {
    console.log("hi frnds i am useCallack()");
  }

  return (
    <div>
      <Demo b={a} />
      <button onClick={() => setCount(count + 1)}>Click {count} </button>
    </div>
  );
}

let Demo = memo( function({ b }) {
  return (
    <div>
      <h1>hi = {b()}</h1>
    </div>
  );
});

export default Calll;
```

→ Q Iss code mei re-rendering ho rahi hai kyu? Function a to change nahi ho raha hai and we have used memo; still there is re-rendering, why?

In the given code, the `Demo` component is re-rendering because the `a` function is **recreated** every time the `Call` component re-renders. This behavior happens because functions defined inside a React component are redefined on each render, which causes their references to change.

Why Does This Cause Re-rendering?

- In React, when a component wrapped in `React.memo` receives a new reference to a prop (even if its value is the same), React considers it as a prop change and re-renders the component.
- Here, the `a` function is passed to `Demo` as the `b` prop. Each time the `Call` component re-renders (due to a state change like `count`), the `a` function is **recreated**, resulting in a new reference.
- `React.memo` compares props using **shallow equality**, and since the new reference of `a` is different from the previous one, `React.memo` cannot prevent `Demo` from re-rendering.

function `a` is getting recreated jab `Call` component re-render hota hai. Function recreation ke karan it is resulting in a new reference even if the function definition is same. Prop change ho raha hai iske karan and re-rendering ho rahi hai.

Ab ise kaise avoid kare? Hum function `a` ko memoize kar sakte hai. Replace that function with `useCallback()`.

let `a = useCallback ( function, [ ] )`

↳ dependency array for state/props.

```
import React, { memo, useCallback, useState } from "react";
```

```
function Calll() {
```

```
  let [count, setCount] = useState(0);
```

```
  let a = useCallback( function(){  
    console.log("hi frnds i am useCallback()");  
  }, []);
```

```
  return (
```

```
    <div>
```

```
      <Demo b={a} />
```

```
      <button onClick={() => setCount(count + 1)}>Click {count} </button>
```

```
    </div>
```

```
  );
```

```
}
```

```
let Demo = memo( function({ b }) {
```

```
  return (
```

```
    <div>
```

```
      <h1>hi = {b()}</h1>
```

```
    </div>
```

```
  );
```

```
})
```

```
export default Calll;
```

↳ Calll.jsx

→ is code mei enton re-rendering nahi hogi.

```
import React, { memo, useCallback, useState } from "react";
```

```
function Calll() {
```

```
  let [count, setCount] = useState(0);
```

```
  let a = useCallback( function(){  
    console.log("hi frnds i am useCallback()");  
  }, [count]);
```

```
  return (
```

```
    <div>
```

```
      <Demo b={a} />
```

```
      <button onClick={() => setCount(count + 1)}>Click {count} </button>
```

```
    </div>
```

```
  );
```

```
}
```

```
let Demo = memo( function({ b }) {
```

```
  return (
```

```
    <div>
```

```
      <h1>hi = {b()}</h1>
```

```
    </div>
```

```
  );
```

```
})
```

```
export default Calll;
```

→ yaha maine prop daal diya hai.

Ab jab jab count change, tab tab a re sender hoga.

